KOI CONTENT PACK

Test Guide

Twelve tests that prove every part of the pack — with the tenant setup each one needs

For anyone comfortable in the Cortex XSIAM console. No content development required.

Pack 1.5.0 · July 2026

CONTENTS

What ships in the pack

A	**KOI integration**	Event collector plus 26 commands — devices, inventory, catalog risk, governance.
B	**Parsing & modeling rules**	Normalise KOI events and map them to the Cortex Data Model.
C	**Alerts dashboard**	Ready-made view of KOI alert activity.
D	**Triage & investigation playbooks**	7 playbooks: triage, item and device investigation, gated response, MCP hunt.
E	**Script Runner playbooks**	5 playbooks plus the KoiScanTracker automation — run KOI scripts fleet-wide.
F	**XQL query library**	72 saved queries: 26 threat hunts and 46 detections.
G	**Test tooling**	An event simulator, so you can test without waiting for real KOI activity.

Every one of these is covered by a test in this guide.

HOW TO USE THIS GUIDE

Every test is laid out the same way

WHAT YOU NEED FIRST

The tenant setup this test depends on — an integration instance, a List, a script in Action Center. If something here is missing the test cannot pass, and it will look like a product fault.

STEPS

What to click, in order, in the Cortex XSIAM console. Commands to paste are shown in monospace.

WHAT TO EXPECT

What a passing test looks like. Where a result commonly looks wrong but is correct, it says so.

Run them in order the first time

Tests 1 to 4 prove data is arriving and correct. Tests 5 to 8 prove the automation. Tests 9 and 10 prove fleet script execution — the only ones needing the Core REST API integration. Tests 11 and 12 are the query library and the simulator, and run any time. Nothing here needs a Cortex XDR integration: XSIAM is the XDR, and its XQL engine is built in.

A test that fails on a missing prerequisite is not a product defect — check the amber panel first.

BEFORE YOU BEGIN

Everything the pack depends on, and where to set it up

What	Where in XSIAM	Needed for
KOI API key	KOI console → Settings → API Access	**Tests 1-8, 11**
KOI integration instance	Settings → Data Sources → Add → KOI	**Tests 1-8, 11**
An egress IP KOI accepts	Run the instance on a Cortex engine if needed	**Tests 1-8, 11**
Core REST API instance	Settings → Data Sources → Add → Core REST API	**Tests 9-10 only**
KOI script in Action Center	Action Center → Scripts Library → upload	**Tests 9-10**
An endpoint group	Tag the agents, then Endpoints → Endpoint Groups → dynamic	**Tests 9-10**
"Koi Script Runner" JSON List	Settings → Object Setup → Lists	**Tests 9-10**
Mail sender instance (optional)	Settings → Data Sources	**Tests 9-10 notifications**

The Core REST API instance is the one people miss. The Script Runner uses it to read endpoint groups larger than 100 — without it the tracker stays empty and every scan run does nothing, which looks exactly like a broken playbook.

INSTALLATION

Two ways in — only one collects events

Method	What lands	Events?
demisto-sdk upload	Everything — integration, playbooks, rules, dashboard	✓ yes
Pack zip upload	Content only. The collector and rules do not activate	✗ no

Ask whoever sent you the pack to install it

Installing with demisto-sdk is a developer task and needs a Standard XSIAM API key. If you are testing rather than deploying, have it installed for you and start at Test 1. Nothing else in this guide needs a developer.

Confirm it landed

- ✓ Settings → Data Sources — a KOI integration is offered.
- ✓ Automation → Playbooks — 12 playbooks whose names start with KOI.
- ✓ Dashboards & Reports — the KOI Alerts Dashboard exists.

TEST PLAN

Twelve tests, four groups

Data is arriving and correct

1 Connectivity

2 Event collection

3 Parsing & data model

4 Dashboard

The automation works

5 Alert triage

6 Item & device investigation

7 Gated response

8 MCP server hunt

Fleet script execution

9 Refresh the tracker

10 Scan due endpoints

Extras, any time

11 Threat-hunting library

12 Simulated test data

! Only tests 9 and 10 need the Core REST API integration. Only tests 1-8 and 11 need a KOI API key. Test 12 needs neither and is the fastest way to see the pack working.

TEST 1

Connectivity and authorisation

PACK CONTENT

- KOI integration

TENANT SETUP

- A KOI API key (KOI console → Settings → API Access)
- Permission to add an integration instance in XSIAM

Steps

1. Settings → Data Sources → Add an instance → search KOI.
2. `Server URL:  https://api.prod.koi.security/`
3. Paste the API key. Optionally set a Tenant Name label.
4. Click Test, then Save.
5. Open the CLI at the bottom of any incident and run:
6. `!koi-devices-list limit=5`

What to expect

- ✓ Test returns Success.
- ✓ The command returns a table of up to five devices.
- ✓ If you set a Tenant Name it will appear on collected events later.

! A 403 here almost always means egress, not a bad key: KOI restricts its sensitive endpoints by source IP. Run the instance on a Cortex engine whose address KOI accepts. A 401 means the key itself.

TEST 2

Event collection

PACK CONTENT

- KOI integration
- Koi Parsing Rule

TENANT SETUP

- Test 1 passing
- Fetch events enabled on the KOI instance

Steps

1. Edit the KOI instance and tick Fetch events.
2. Choose which event types to collect, then Save.
3. Wait for two collection cycles.
4. Open Incident Response → Investigation → Query Builder and run:

```
dataset = koi_koi_raw
| comp count() as rows by source_log_type
```

What to expect

- ✓ Rows appear under Alerts, Audit, or both.
- ✓ Counts grow between cycles.

! No rows at all usually means fetch was only just enabled — give it two cycles. Audit missing on its own means the Audit types filter is narrowed; leave it empty for all types.

TEST 3

Parsing and the data model

PACK CONTENT

- Koi Parsing Rule
- Koi Modeling Rule

TENANT SETUP

- Test 2 passing — events present in koi_koi_raw

Steps

1. In Query Builder, count real alerts rather than rows:
2. `dataset = koi_koi_raw`
3. `| filter source_log_type = "Alerts"`
4. `| alter nid = json_extract_scalar(metadata, "$.notification_event_id")`
5. `| comp count() as rows, count_distinct(nid) as real_alerts`

What to expect

- ✓ Both numbers return.
- ✓ rows is larger than real_alerts — often much larger.
- ✓ Fields such as item_id, alert_hostname and risk_level are populated and directly queryable.

! rows far exceeding real_alerts is CORRECT. KOI re-sends every still-open alert on each fetch, so a row is a delivery, not an alert. Anything counting alerts must count distinct notification ids — the dashboard already does.

TEST 4

Dashboard

PACK CONTENT

- Koi Alerts Dashboard
- Koi Parsing Rule

TENANT SETUP

- Test 2 passing — events present

Steps

1. Dashboards & Reports → open the KOI Alerts Dashboard.
2. Read the alert totals, risk mix and top items.
3. Come back after another collection cycle and compare.

What to expect

- ✓ Widgets render with data.
- ✓ Alert counts are far lower than the raw row count from Test 3.
- ✓ Counts stay stable as fetches repeat — they do not climb on their own.

! A widget whose number climbs steadily while nothing new happens in KOI would be counting deliveries rather than alerts. Every widget in this pack de-duplicates first.

TEST 5

Alert triage

PACK CONTENT

- KOI IR - Alert Triage
- KOI IR - Extract Alert Context
- KOI IR - Investigate Item / Device
- incident field: KOI Notification ID

TENANT SETUP

- Test 1 passing — the KOI integration answers
- A KOI alert in XSIAM (a correlation rule over koi_koi_raw, or Test 12 for simulated data)

Steps

1. Open a KOI alert.
2. Attach the triage playbook, or run from the CLI:
3. `!setPlaybook playbookId="KOI IR - Alert Triage"`
4. Watch the Work Plan, then read the War Room.

What to expect

- ✓ Context is extracted: item, marketplace, risk, host.
- ✓ A verdict is posted: Malicious, Suspicious or Benign.
- ✓ Low risk auto-closes; critical and high escalate.
- ✓ Medium and pending stay open for a human.

! Everything coming out Suspicious usually means the alert carried no KOI detail for the playbook to read. Check the correlation rule passes the KOI fields through. Pending means KOI has not finished scoring — it deliberately never auto-closes.

TEST 6

Item and device investigation

PACK CONTENT

- KOI IR - Investigate Item
- KOI IR - Investigate Device
- KOI IR - Enrich Item

TENANT SETUP

- Test 1 passing
- An item id and its marketplace, and a device id — both visible on any KOI alert

Steps

1. Automation → Playbooks → KOI IR - Investigate Item → Run.
2. Supply item_id and marketplace.
3. Repeat with KOI IR - Investigate Device and a device id.

What to expect

- ✓ An investigation summary with catalog risk and an AI-written summary.
- ✓ Organisation exposure: installs, endpoints, affected users.
- ✓ Both governance counts — on blocklist AND on allowlist.
- ✓ For the device: everything installed on that host, and which of it is risky.

! Both governance counts matter. Zero blocklist hits alone does not mean nobody has governed the item — it may have been explicitly allowed, and blocking it would override that decision.

TEST 7

Gated response

PACK CONTENT

- KOI IR - Block and Remediate
- KOI IR - Investigate Item

TENANT SETUP

- Test 1 passing
- An item id and marketplace
- Permission to approve a task

Steps

1. Automation → Playbooks → KOI IR - Block and Remediate → Run.
2. Supply item_id and marketplace.
3. Open the pending approval task and read it.
4. Approve, or leave it pending.

What to expect

- ✓ The run PARKS on an approval task and waits.
- ✓ The approval shows the full investigation and both governance counts.
- ✓ Nothing reaches the blocklist until a human approves.
- ✓ An item already blocked short-circuits instead of being added twice.

! This is the test to repeat after any change to the response playbook. If a run ever completes without stopping for approval, stop and investigate before using the pack in production.

TEST 8

MCP server hunt

PACK CONTENT
- KOI Hunting - MCP Server Audit

TENANT SETUP
- Test 1 passing — the KOI integration answers

Steps

1. Automation → Playbooks → KOI Hunting - MCP Server Audit → Run.
2. Optionally set risk_levels (default high, critical).
3. Read the War Room table.
4. To schedule it: Automation → Jobs → New Job, time-triggered.

What to expect

- ✓ A table of MCP servers at or above the risk threshold.
- ✓ An empty table is a valid result — it means nothing risky is installed.

! This asks KOI what it has already scored. It finds risky agentic-AI assets without waiting for an alert about them.

TEST 9

Script Runner — refresh the tracker

PACK CONTENT

- KOI Script Runner - Refresh Job
- KOI Script Runner - Refresh Entry
- KoiScanTracker (our automation — not the KOI script)
- List: Koi Script Runner

TENANT SETUP

- Core REST API instance ← the one people miss
- KOI deployment script in Action Center — download it from YOUR KOI console first (KOI's script, not a Cortex one; no parameters)
- An endpoint group — tag the agents, then use a dynamic group
- The Koi Script Runner List created (step 1)

Steps

1. Settings → Object Setup → Lists → New List, type JSON, named exactly Koi Script Runner.
2. Open Lists/README.md in the pack and copy the JSON block — it is formatted ready to paste (do NOT copy from list-Koi_Script_Runner.json; its data field is escaped).
3. Edit two things only: endpoint_groups (your group) and script.name (the KOI script exactly as it appears in Action Center). One entry per OS.
4. Everything else — tracker_list, rescan_interval_hours, max_endpoints — already has working defaults.
5. Automation → Jobs → New Job → time-triggered → playbook KOI Script Runner - Refresh

What to expect

- ✓ A tracker List appears under Lists, named as in your entry.
- ✓ It fills with one row per endpoint: an id and a last-scan value.
- ✓ Re-running adds new endpoints without disturbing existing rows.

! Without a Core REST API instance this test cannot pass — the refresh reads endpoint groups through it. The tracker simply stays empty, which looks like a broken playbook.

TEST 10

Script Runner — scan due endpoints

PACK CONTENT

- KOI Script Runner - Scan Job
- KOI Script Runner - Process Config Entry
- KOI Script Runner - Execute Endpoint Script
- KoiScanTracker (our automation)

TENANT SETUP

- Test 9 passing — the tracker List has rows
- Connected agents in the group, matching the entry's endpoint_os

Steps

1. Automation → Jobs → New Job → time-triggered → playbook KOI Script Runner - Scan Job.
2. Enable the Job, then Run now.
3. Open the run, then check Action Center.
4. Run it a second time immediately.

What to expect

- ✓ The script is dispatched to due, connected endpoints.
- ✓ Those endpoints get a last-scan timestamp in the tracker.
- ✓ Offline, wrong-OS and unknown endpoints stay due and retry later.
- ✓ The second run does nothing — everything is inside the rescan interval.

! SKIPPED means nothing is due right now and sends no email, by design. STILL RUNNING means the script was dispatched and Action Center is finishing on its own. Neither is a failure.

TEST 11

Threat-hunting query library

PACK CONTENT

- none — the queries are files you paste

TENANT SETUP

- Test 2 passing — events present
- Nothing else — the cross-dataset hunts read xdr_data, which XSIAM holds natively

Steps

1. Open docs/xql in the pack you were sent.
2. Read QUERY_LIBRARY.md first — it explains the rules.
3. Paste a hunt into Query Builder, starting with H2.1.
4. Try a detection too, for example A1.

What to expect

- ✓ Each query runs and returns rows, or a clean empty result.
- ✓ Hunts surface findings KOI scored that nobody actioned.
- ✓ 26 hunts and 46 detections are included.

! These read raw event fields, so they keep working regardless of the modeling rule. An empty result is a valid answer, not a failure.

TEST 12

Simulated test data

PACK CONTENT

- KoiSimulateEvents (test automation)

TENANT SETUP

- The KoiSimulateEvents automation imported (ships in TestTools)
- No KOI API key and no Core REST API needed

Steps

1. Open the CLI on any incident.
2. Run:
3. `!KoiSimulateEvents alerts=40 duplicate_factor=8 audit=60`
4. Read the summary it prints — it tells you the answers in advance.
5. Query koisim_koisim_raw to check them.

What to expect

- ✓ 380 events land in koisim_koisim_raw.
- ✓ 40 distinct alerts delivered 8 times each, as real KOI does.
- ✓ The counts you measure match the summary exactly.

! It writes to its own dataset, never koi_koi_raw, so it cannot disturb real data. Use it to see the pack working before real KOI activity exists.

IF SOMETHING FAILS

Check the prerequisite before the product

403 on KOI commands	Source IP not accepted by KOI	Run the instance on a Cortex engine KOI accepts
401 on KOI commands	The API key itself	Re-create the key with the xt-Administrator role
Engine is not connected	The Cortex engine is offline	Every koi- command fails before reaching KOI. Restart the engine
No events after enabling fetch	Only one cycle has run	Wait two cycles, then re-check Test 2
Everything triages as Suspicious	The alert carried no KOI detail	Check the correlation rule passes KOI fields through
Tracker List stays empty	No Core REST API instance	Configure it — Test 9 cannot pass without it
Scan run does nothing	Refresh has not run yet	Run the Refresh job first; an empty tracker means nothing is due
A Job never fires	It was created disabled	Enable it and confirm a next-run time is shown

Six of these eight are tenant setup, not the pack. Check the amber panel on the test before raising a defect.

SIGN-OFF

One line of evidence per test

1	Test returns Success; devices list returns rows
2	koi_koi_raw grows between collection cycles
3	Distinct alerts far below raw rows — and that is correct
4	Dashboard renders; counts stay stable across fetches
5	A verdict is reached; low risk auto-closes
6	Investigation shows catalog risk and both governance counts
7	Response parks on approval; blocklist untouched until approved
8	MCP audit returns servers at or above the threshold
9	Tracker List is created and fills with endpoints
10	Scan marks only due, connected endpoints; re-run is a no-op
11	Library queries run and return results
12	Simulated counts match the summary exactly

All twelve green — the pack is ready for production use.

Full detail in KOI_Integration_Customer_Guide_v1.4.0. Diagnostics in the troubleshooting guide.